

llama.cpp CUDA Server i Docker

Sådan kører du llama-server med GPU-acceleration
i en Docker-container

CUDA 12 | Docker | NVIDIA | llama.cpp

Indhold

1. Forudsætninger
2. Metode A: Officielt prebuilt image (anbefalet)
3. Metode B: Byg selv med CUDA i Docker
4. Download en GGUF-model
5. Kør llama-server
6. Test serveren
7. Tips og tunings
8. Fejlfinding

1. Forudsætninger

Før du starter, skal din maskine have følgende:

- NVIDIA GPU med driver installeret
- Docker Engine (docker compose er ikke nødvendigt)
- NVIDIA Container Toolkit (nvidia-container-toolkit)

Tjek at alt er klar:

```
nvidia-smi
docker info | grep -i nvidia
nvidia-ctk --version
```

i Har du ikke nvidia-container-toolkit? Installer med:

```
sudo apt install nvidia-container-toolkit
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker
```

2. Metode A: Officielt prebuilt image

Den nemmeste metode — brug det officielle llama.cpp image med CUDA 12 færdigbygget:

2.1 Træk image

```
docker pull ghcr.io/ggml-org/llama.cpp:cu12-latest
```

2.2 Kør serveren

Opret en mappe til dine modeller og kør:

```
mkdir -p ~/models

docker run -d --gpus all \
  -p 8080:8080 \
  -v ~/models:/models \
  --name llama-server \
  ghcr.io/ggml-org/llama.cpp:cu12-latest \
  llama-server --host 0.0.0.0 --port 8080 \
  -m /models/din-model.gguf -ngl 99
```

i --gpus all giver containeren adgang til alle GPU'er.

-v ~/models:/models binder din models-mappe ind i containeren.

-ngl 99 offloader så mange layers som muligt til GPU.

3. Metode B: Byg selv med CUDA

Hvis du vil bygge llama.cpp fra source med præcis de flags du ønsker:

3.1 Opret en Dockerfile

```
FROM nvidia/cuda:12.4.1-devel-ubuntu22.04

RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential cmake git \
    && rm -rf /var/lib/apt/lists/*

RUN git clone --depth 1 https://github.com/ggml-org/llama.cpp \
    && cd llama.cpp \
    && cmake -B build -DGGML_CUDA=ON -DCMAKE_BUILD_TYPE=Release \
    && cmake --build build -j$(nproc) --target llama-server

ENV PATH="/llama.cpp/build/bin:$PATH"

EXPOSE 8080

ENTRYPOINT ["llama-server"]
```

3.2 Byg og kørs

```
docker build -t llama-cuda .

docker run -d --gpus all \
    -p 8080:8080 \
    -v ~/models:/models \
    --name llama-server \
    llama-cuda \
    --host 0.0.0.0 --port 8080 \
    -m /models/din-model.gguf -ngl 99
```

4. Download en GGUF-model

Du kan hente modeller fra HuggingFace. Her er nogle gode bud:

- Mistral 7B: TheBloke/Mistral-7B-Instruct-v0.2-GGUF
- Llama 3.2 3B: bartowski/Llama-3.2-3B-Instruct-GGUF
- Qwen 2.5 7B: Qwen/Qwen2.5-7B-Instruct-GGUF
- DeepSeek R1 Distil: bartowski/DeepSeek-R1-Distill-Qwen-7B-GGUF

Download direkte på værtsmaskinen:

```
cd ~/models

# Installer huggingface-cli
pip install huggingface-hub

# Hent en model (eksempel: Llama 3.2 3B, Q4_K_M)
huggingface-cli download bartowski/Llama-3.2-3B-Instruct-GGUF \
  --local-dir ./llama-3.2-3b \
  --include "*Q4_K_M*"
```

5. Kør llama-server

Når du har en GGUF-model, start serveren:

```
docker run -d --gpus all \
  --name llama-server \
  -p 8080:8080 \
  -v ~/models:/models \
  ghcr.io/ggml-org/llama.cpp:cu12-latest \
  llama-server \
  --host 0.0.0.0 \
  --port 8080 \
  -m /models/llama-3.2-3b/llama-3.2-3b.Q4_K_M.gguf \
  -ngl 99 \
  -c 4096 \
  --mlock
```

Forklaring af flags:

- -ngl 99: Offload så mange layers til GPU som muligt
- -c 4096: Kontekstlængde (antal tokens)
- --mlock: Lås modellen i RAM (forhindrer swapping)
- --no-mmap: Brug hvis modellen er på et network-filesystem

6. Test serveren

Når serveren kører, test den med curl:

```
# Tjek at serveren er oppe
curl http://localhost:8080/health

# Send en chat completion
curl -X POST http://localhost:8080/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{
  "messages": [
    {"role": "user", "content": "Hej! Fortæl mig en kort vittighed."}
  ],
  "temperature": 0.7,
  "max_tokens": 200
}'
```

Du kan også tjekke logs:

```
docker logs llama-server

# Se GPU-brug
nvidia-smi
```

7. Tips og tunings

GPU-indstillinger

Juster -ngl efter din GPU-hukommelse:

- GTX 1080 Ti (11 GB): -ngl 99 for 7B modeller, -ngl 20-30 for 13B
- RTX 3090 (24 GB): -ngl 99 for op til 13B modeller
- Mindre VRAM: Brug Q4_K_M eller Q3_K_M kvantisering

Kontekststørrelse

Juster -c (kontekstlængde) efter dit behov:

- -c 2048: Standard, lavt memoryforbrug
- -c 4096: God balance for de fleste opgaver
- -c 8192: Store dokumenter, kræver mere VRAM

Genstart med nye indstillinger

```
docker stop llama-server
docker rm llama-server
# Kør docker run igen med nye flags
```

Brug docker-compose (avanceret)

Opret docker-compose.yml:

```
services:
  llama:
    image: ghcr.io/ggml-org/llama.cpp:cu12-latest
    ports:
      - "8080:8080"
    volumes:
      - ~/models:/models
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: all
              capabilities: [gpu]
    command: >
      llama-server --host 0.0.0.0 --port 8080
      -m /models/din-model.gguf -ngl 99 -c 4096
```

8. Fejlfinding

Problem	Løsning
Could not find CUDA	Manglende nvidia-container-toolkit. Installer det.
cuBLAS not found	Brug prebuilt image: ghcr.io/ggml-org/llama.cpp:cu12-latest
Out of memory	Brug mindre model eller lavere kvantisering (Q4_K_M -> Q3_K_M)
--gpus all virker ikke	Tjek: nvidia-smi virker? nvidia-ctk kørt? Docker genstartet?
Langsom generation	Tjek -ngl flag. For lidt layers på GPU.
Port 8080 i brug	Skift til anden port: -p 8081:8080